

Name:

CS403/503 Programming Languages
Spring 2025
Assignment #1

1. Problem #3 (i.e., #3 of the Problem Set) of Chapter 3 on Page 157

Answer:

$$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$$
$$\langle \text{id} \rangle \rightarrow A \mid B \mid C$$
$$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle * \langle \text{term} \rangle \mid \langle \text{term} \rangle$$
$$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle + \langle \text{term} \rangle \mid \langle \text{factor} \rangle$$
$$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$$

2. Problem #8 of Chapter 3 on Page 158

Answer:

To prove this grammar is ambiguous we find a string that has more than one distinct parse tree. A suitable candidate is “a + b + c”.

Left-associative grouping:

$$\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle$$
$$\langle A \rangle \rightarrow (\langle A \rangle + \langle A \rangle) + \langle A \rangle$$
$$\langle A \rangle \rightarrow (a + b) + c$$

Right-associative grouping:

$$\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle$$
$$\langle A \rangle \rightarrow \langle A \rangle + (\langle A \rangle + \langle A \rangle)$$
$$\langle A \rangle \rightarrow a + (b + c)$$

Since both derivations lead to different parse trees, the grammar does not enforce a unique structure for the same string.

3. Problem #20 of Chapter 3 on Page 159

Name:

Answer:

1. Syntax rule: $\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

Semantic rule: $\langle \text{expr} \rangle.\text{expected_type} \leftarrow \langle \text{id} \rangle.\text{actual_type}$

2. Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle$

Semantic rule:

$\langle \text{expr} \rangle.\text{actual_type} \leftarrow$ if $(\langle \text{id} \rangle.\text{actual_type} = \text{int})$ and $(\langle \text{expr} \rangle.\text{actual_type} = \text{int})$ then int else real end if

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

3. Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle * \langle \text{expr} \rangle$

Semantic rule:

$\langle \text{expr} \rangle.\text{actual_type} \leftarrow$ if $(\langle \text{id} \rangle.\text{actual_type} = \text{int})$ and $(\langle \text{expr} \rangle.\text{actual_type} = \text{int})$ then int else real end if

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

4. Syntax rule: $\langle \text{expr} \rangle \rightarrow (\langle \text{expr} \rangle)$

Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{expr} \rangle.\text{actual_type}$

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

5. Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle$

Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{id} \rangle.\text{actual_type}$

Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$

6. Syntax rule: $\langle \text{id} \rangle \rightarrow A \mid B \mid C$

Semantic rule: $\langle \text{id} \rangle.\text{actual_type} \leftarrow \text{look-up}(\langle \text{id} \rangle.\text{string})$

Name:

4. Problem #7 of Chapter 4 on Page 194

Answer:

Consider the traditional grammar for arithmetic expressions that follows:

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow id$

State	Action						Goto		
	id	+	*	(	)	\$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

String: $id * (id + id)$

Stack	Input	Action
0	$id * (id + id)$$	Shift 5
0id5	$* (id + id)$$	Reduce 6 [0, F]
0F3	$* (id + id)$$	Reduce 4 [0, T]
0T2	$* (id + id)$$	Shift 7
0T2*7	$(id + id)$$	Shift 4
0T2*7(4	$id + id)$$	Shift 5
0T2*7(4id5	$+ id)$$	Reduce 6 [4, F]
0T2*7(4F3	$+ id)$$	Reduce 4 [4, T]
0T2*7(4T2	$+ id)$$	Reduce 2 [4, E]
0T2*7(4E8	$+ id)$$	Shift 6

Name:

0T2*7(4E8+6	id)\$	Shift 5
0T2*7(4E8+6id5	)\$	Reduce 6 [6, F]
0T2*7(4E8+6F3	)\$	Reduce 4 [6, T]
0T2*7(4E8+6T9	)\$	Reduce 1 [4, E]
0T2*7(4E8	)\$	Shift 11
0T2*7(4E8)11	\$	Reduce 5 [7, F]
0T2*7F10	\$	Reduce 3 [0, T]
0T2	\$	Reduce 2 [0, E]
0E1	\$	accept